

ML for Health

Laboratory # 6: Gaussianity, ICA and EEG

Prof. Monica Visintin - Politecnico di Torino

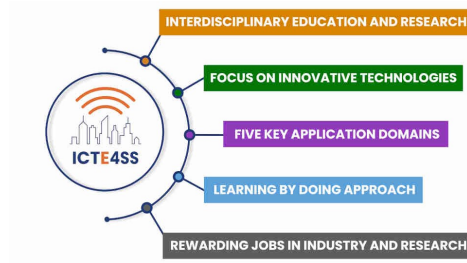


Table of Contents

Gaussianity tests

An artificial example for ICA

EEGLAB

ICA in EEGLAB

Generation of a Gaussian random variable

There are two basic methods to generate samples drawn from a Gaussian distribution:

1. Apply a nonlinearity to a couples of statistical independent uniform random variables (Box-Muller method)
2. Exploit the central limit theorem

Gaussian distribution using Box-Muller method [1]

1. We start from the analysis of two random Gaussian variables G_1 and G_2 seen as a point in $\mathbb{R}^2$: a point with cartesian coordinates (G_1, G_2) has polar coordinates (R, Θ) where the the radius R and the angle Θ are

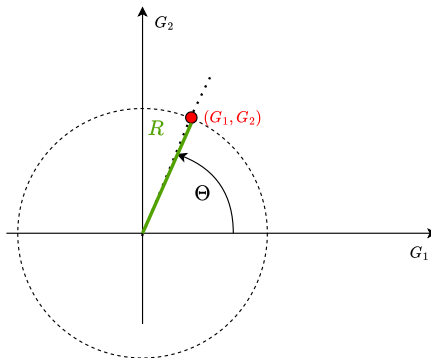
$$R = \sqrt{G_1^2 + G_2^2}, \quad \Theta = \tan^{-1}(G_2/G_1)$$

Note that the equations can be inverted to get G_1 and G_2 from R and Θ :

$$G_1 = R \cos(\Theta), \quad G_2 = R \sin(\Theta) \tag{1}$$

Since it is easier to generate R and Θ (their cumulative distribution function, CDF, has a simpler mathematical expression), we will generate R and Θ from uniformly distributed random variables, then we will generate the Gaussian R.V. from R and Θ using eqn. (1).

Gaussian distribution using Box-Muller method [2]



Gaussian distribution using Box-Muller method [3]

- 1.1** Let us evaluate the cumulative distribution function (CDF) of the random variable R , when G_1 and G_2 are Gaussian, statistically independent, with zero mean and variance σ^2 :

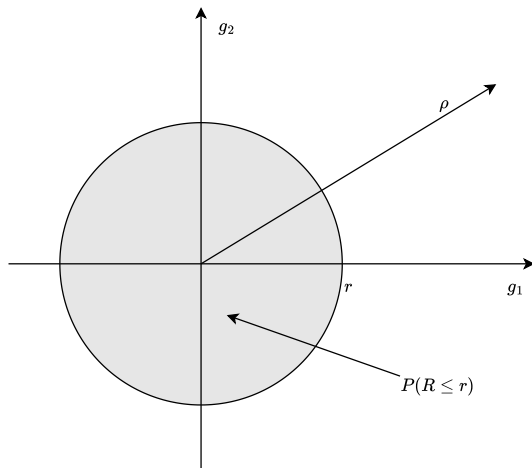
$$F_R(r) = P(R \leq r) = P(\text{point } (G_1, G_2) \text{ is inside circle } C \text{ with radius } r), \quad r \geq 0$$

$$F_R(r) = \int_C f_{G_1, G_2}(g_1, g_2) dg_1 dg_2$$

where the joint probability density function of G_1 and G_2 is

$$f_{G_1, G_2}(g_1, g_2) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(g_1^2 + g_2^2)}{2\sigma^2}\right)$$

Gaussian distribution using Box-Muller method [4]



Gaussian distribution using Box-Muller method [5]

The integral is more easily evaluated in polar coordinates, where $g_1^2 + g_2^2 = \rho^2$ and $dg_1 dg_2$ becomes $\rho d\rho d\theta$:

$$F_R(r) = \int_{\rho \leq r} \int_{\theta \in [-\pi, \pi]} \frac{1}{2\pi\sigma^2} \exp\left(-\frac{\rho^2}{2\sigma^2}\right) \rho d\rho d\theta$$

$$F_R(r) = \frac{1}{2\pi\sigma^2} 2\pi \int_{\rho \leq r} \rho \exp\left(-\frac{\rho^2}{2\sigma^2}\right) d\rho = \frac{1}{\sigma^2} \int_{\rho \leq r} \rho \exp\left(-\frac{\rho^2}{2\sigma^2}\right) d\rho$$

Note that:

$$\frac{d}{d\rho} \exp\left(-\frac{\rho^2}{2\sigma^2}\right) = -\frac{2\rho}{2\sigma^2} \exp\left(-\frac{\rho^2}{2\sigma^2}\right) = -\frac{1}{\sigma^2} \rho \exp\left(-\frac{\rho^2}{2\sigma^2}\right)$$

Therefore

$$F_R(r) = -\exp\left(-\frac{\rho^2}{2\sigma^2}\right) \Big|_0^r = 1 - \exp\left(-\frac{r^2}{2\sigma^2}\right), \quad r \geq 0.$$

The probability density function (pdf) of R is the derivative of $F_R(r)$:

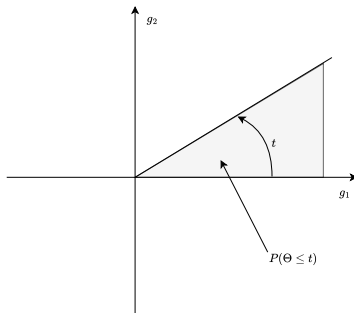
$$f_R(r) = \frac{r}{\sigma^2} \exp\left(-\frac{r^2}{2\sigma^2}\right), \quad r \geq 0.$$

The random variable R with the above CDF and pdf is called **Rayleigh** random variable.

Gaussian distribution using Box-Muller method [6]

1.2 As for the angle Θ :

$$F_{\Theta}(t) = P(\Theta \leq t) = \int_{\rho \leq \infty} \int_{\theta \leq t} \frac{1}{2\pi\sigma^2} \exp\left(-\frac{\rho^2}{2\sigma^2}\right) \rho d\rho d\theta, \quad t \in [0, 2\pi]$$



Gaussian distribution using Box-Muller method [7]

$$F_{\Theta}(t) = P(\Theta \leq t) = \int_{\theta \leq t} \frac{1}{2\pi} \left(\int_{\rho \leq \infty} f_R(\rho) d\rho \right) d\theta = \int_{\theta \leq t} \frac{1}{2\pi} d\theta,$$

since the integral from 0 to infinity of the Rayleigh pdf is 1. Thus, the CDF of Θ is

$$F_{\Theta}(t) = \frac{t}{2\pi}, \quad t \in [0, 2\pi]$$

The above formula means that Θ is uniformly distributed in $[0, 2\pi]$:

$$f_{\theta}(t) = \frac{1}{2\pi}, \quad t \in [0, 2\pi]$$

Gaussian distribution using Box-Muller method [8]

2. We can exploit these results to generate a couple of independent Gaussian random variables. As already discussed in the lecture, any computer/programming language is able to generate samples drawn from a uniform random variable $U \sim \mathcal{U}(0, 1)$.

2.1 Let $U_1 \sim \mathcal{U}(0, 1)$, then

$$\Theta = 2\pi U_1 \sim \mathcal{U}(0, 2\pi).$$

Gaussian distribution using Box-Muller method [9]

2.2 Let $U_2 \sim \mathcal{U}(0, 1)$. Now we must generate the Rayleigh random variable R , through a nonlinear transformation $f(\cdot)$ of U_2 :

$$F_R(r) = P(R \leq r) = P(f(U_2) \leq r) = P(U_2 \leq f^{-1}(r)) = f^{-1}(r) = F_R(r)$$

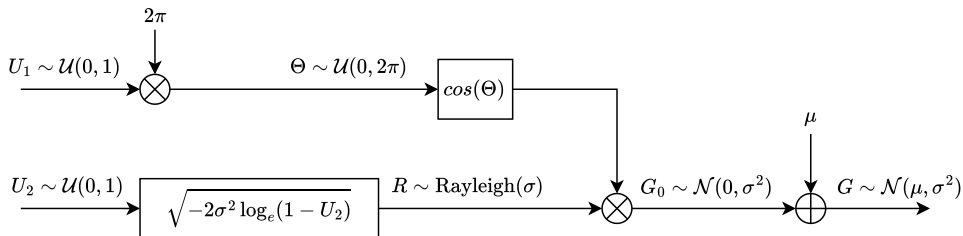
$$f(u) = F_R^{-1}(u)$$

$$y = 1 - \exp\left(-\frac{x^2}{2\sigma^2}\right) \Leftrightarrow x = \sqrt{-2\sigma^2 \log(1 - y)}$$

$$f(u) = \sqrt{-2\sigma^2 \log(1 - u)}, \quad \Rightarrow \quad R = \sqrt{-2\sigma^2 \log(1 - U_2)}$$

The random variable R is Rayleigh distributed (with the scale parameter σ): $R \sim \text{Rayleigh}(\sigma)$.

Gaussian distribution using Box-Muller method [10]



Gaussian distribution using Box-Muller method [11]

3. To draw Gaussian samples (variance σ^2):

3.1 Generate array $\mathbf{u}_1$ with N_s values drawn from $U_1 \sim \mathcal{U}(0, 1)$; generate

$$\boldsymbol{\theta} = 2\pi\mathbf{u}_1,$$

which stores N_s samples drawn from $\Theta \sim \mathcal{U}(0, 2\pi)$.

3.2 Generate array $\mathbf{u}_2$ with N_s values drawn from $U_2 \sim \mathcal{U}(0, 1)$; generate array

$$\mathbf{r} = \sqrt{-2\sigma^2 \log(1 - \mathbf{u}_2)},$$

which stores N_s samples drawn from $R \sim \text{Rayleigh}(\sigma)$.

3.3 Generate array

$$\mathbf{g}_1 = \mathbf{r} \cos(\boldsymbol{\theta}),$$

which stores N_s samples drawn from $G_1 \sim \mathcal{N}(0, \sigma^2)$. If you need a second Gaussian random variable $G_2 \sim \mathcal{N}(0, \sigma^2)$ (stat. indep. of G_1), generate the array $\mathbf{g}_2 = \mathbf{r} \sin(\boldsymbol{\theta})$.

3.4 If you want a mean $\mu \neq 0$, then generate

$$\mathbf{g} = \mathbf{r} \cos(\boldsymbol{\theta}) + \mu,$$

which stores samples from $G \sim \mathcal{N}(\mu, \sigma^2)$

Gaussian distribution using central limit theorem [1]

The central limit theorem

Given N i.i.d. (independent and identically distributed) random variables X_i , with mean zero and variance σ^2 , then

$$Z = \frac{1}{\sqrt{N}} \sum_{i=1}^N X_i$$

tends to have a Gaussian probability density function with zero mean and variance σ^2 , as N grows to infinity.

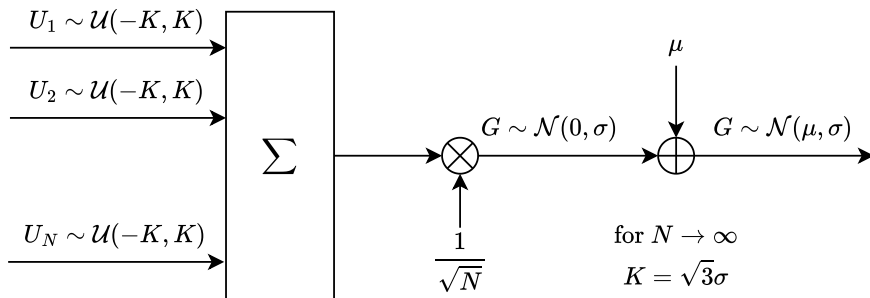
We can use N i.i.d. uniformly distributed random variables $X_i \sim \mathcal{U}(-K, K)$ (note that they must have zero mean). The variance of X_i is

$$\sigma^2 = \int_{-K}^K x^2 f_{X_i}(x) dx = 2 \int_0^K x^2 \frac{1}{2K} dx = \frac{1}{K} \left. \frac{x^3}{3} \right|_0^K = \frac{1}{K} \frac{K^3}{3} = \frac{K^2}{3}.$$

To obtain the desired variance of Z , it is necessary to set

$$K = \sqrt{3} \sigma$$

Gaussian distribution using central limit theorem [2]



Gaussian distribution using central limit theorem [3]

1. Generate the 2D array/matrix $\mathbf{u}$ with N_s rows and N columns and values drawn from $U_1 \sim \mathcal{U}(0, 1)$; generate

$$\mathbf{w} = \sqrt{3} \sigma \, 2(\mathbf{u} - 0.5),$$

which stores $N_s \times N$ samples drawn from $W \sim \mathcal{U}(-\sqrt{3}\sigma, \sqrt{3}\sigma)$.

2. Generate

$$\mathbf{z} = \frac{1}{\sqrt{N}} \sum_{i=1}^N \mathbf{w}[:, i], \quad \text{sum along the } N \text{ columns}$$

The obtained array stores N_s samples with approximately Gaussian distribution $\mathcal{N}(0, \sigma^2)$.

3. If you want a mean $\mu \neq 0$, then generate

$$\mathbf{g} = \mathbf{z} + \mu,$$

which approximately stores samples from $G \sim \mathcal{N}(\mu, \sigma^2)$

The approximation tends to be truly Gaussian if N becomes very large.

Gaussian distribution using central limit theorem [4]

How large should N be? Let us check the Gaussianity of the array.

Tests [1]

1. In the Python script you find in DropBox, N_s samples of a truly (using the Box-Muller method) or approximate (using the Central Limit Theorem) Gaussian random variable are generated and stored in array $\mathbf{x}$. It is possible to set the desired values of σ and μ . **For the case of Central Limit Theorem, it is possible to set the value of N (number of uniform random variables to sum).** We want to see if the N_s samples stored in array $\mathbf{x}$ pass the tests, so that we cannot reject the fact that they are samples from a Gaussian random variable with mean μ and variance σ .
2. The normalized **histogram** of $\mathbf{x}$ is compared to the theoretical Gaussian pdf. To plot the histogram, the **Scott's rule** is used to set the bin width.
3. The **cumulative distribution function** is measured and compared to the theoretical one.

Tests [2]

4. The **normal probability plot** is generated using $n = 20$ quantiles. The measured quantiles are plotted against the quantiles of the *normalized* Gaussian random variable (mean zero and variance 1), so that it is possible to measure
 - ▶ the mean of the generated samples as the value on the y-axis corresponding to value 0 on the x-axis,
 - ▶ the standard deviation as the slope of the line.

The ideal quantiles are also shown (for the known values of μ and σ) to better compare theory and measurements.

5. The **t-score** is evaluated (remember that in the case of t-score, there is no assumption on the value of the variance, while we test the fact that the samples were drawn from a Gaussian random variable with mean μ):

$$t_{meas} = \frac{m - \mu}{S/\sqrt{N}}; \quad m = \frac{1}{N_s} \sum_{k=0}^{N_s-1} x[k], \quad S = \frac{1}{N_s - 1} \sum_{k=0}^{N_s-1} (x[k] - m)^2.$$

To test the hypothesis, we must evaluate the **two-tail p-value** for the found value of t_{meas} :

$$p = P(|T| > t_{meas})$$

Tests [3]

where T is a student's T random variable with degrees of freedom $N_s - 1$. In the case of t-score, evaluation of the p-value p can be done "exactly" (apart from numerical errors) using the scipy library:

```
1 from scipy.stats import t
2 p_val = 2*(1-t.cdf(t_meas, Nsamp-1))
```

Note that the student's t random variable has a symmetric pdf around 0, therefore

$$\begin{aligned} p &= P(|T| > t_{meas}) = P(T > t_{meas}) + P(T < -t_{meas}) \\ &= 2P(T > t_{meas}) = 2(1 - P(T \leq t_{meas})) = 2(1 - F_T(t_{meas})) \end{aligned}$$

where $F_T(t) = P(T \leq t)$ is the cumulative distribution function (CDF, cdf for scipy) of T .

Tests [4]

6. The **excess kurtosis** is measured using the formula

$$k_{e, meas} = \frac{N_s - 1}{(N_s - 2)(N_s - 3)} [(N_s + 1)k_0 - 3(N_s - 1)]$$

$$k_0 = \frac{\frac{1}{N_s} \sum_{i=1}^{N_s} (x_i - m)^4}{\left(\frac{1}{N_s} \sum_{i=1}^{N_s} (x_i - m)^2 \right)^2}, \quad m = \frac{1}{N_s} \sum_{i=1}^{N_s} x_i$$

In this case, we do not have a theoretical CDF for the excess kurtosis of N_s samples drawn from a Gaussian R.V., therefore we need to measure it by running N_{exp} experiments (for example $N_{exp} = 10000$) in which we generate samples truly drawn from a Gaussian random variable with mean μ and standard deviation σ (we assume that numpy generation of Gaussian random variables is "perfect") and we measure the excess kurtosis. From the obtained N_{exp} values of excess kurtosis, we obtain the CDF of $|K_e|$ and we get the p-value.

Tests [5]

7. Finally we measure the value of the **Anderson-Darling test**:

$$a_{meas}^2 = -N_s - \sum_{i=1}^{N_s} \frac{2i-1}{N_s} [\ln F(y_i) + \ln(1 - F(y_{N_s+1-i}))]$$

where y_i is the i -th term when the values $\{x_i\}_{i=1}^{N_s}$ are sorted from smallest to largest, whereas y_{N_s+1-i} is the i -th term when the values $\{x_i\}_{i=1}^{N_s}$ are sorted from largest to smallest; $F(u)$ is the cumulative distribution function of the desired random variable $x \sim \mathcal{N}(\mu, \sigma)$. Even in this case, we have no theoretical formula for the CDF of the random variable a^2 , and we resort again to running N_{exp} experiments, as for the case of excess kurtosis.

8. Having set the **significance level** to $\alpha = 0.05$, we reject the Gaussian hypothesis (at significance level α) if the p-value p is smaller than α . If the p-value is larger than α , then we cannot say anything (the Gaussian hypothesis might be correct or might be wrong).

To do

1. Run the script using the Box-Muller method to generate $N_s = 100, 500, 1000$ samples. Is there a case in which the test fails and you reject the Gaussian hypothesis (i.e. you think that the Box-Muller generation method is not correct, because you do not get samples from a Gaussian random variable with the desired mean and variance)?
2. Run again the script exploiting the central limit theorem with $N_s = 1000$ samples and $N = 1, 2, 3, 4, \dots$ uniform random variables. Which value of N "allows you to get a Gaussian random variable"? better, for which value of N you do not reject the Gaussian hypothesis?

Change the seed of numpy and repeat the experiment. Does the minimum value of N change?

Table of Contents

Gaussianity tests

An artificial example for ICA

EEGLAB

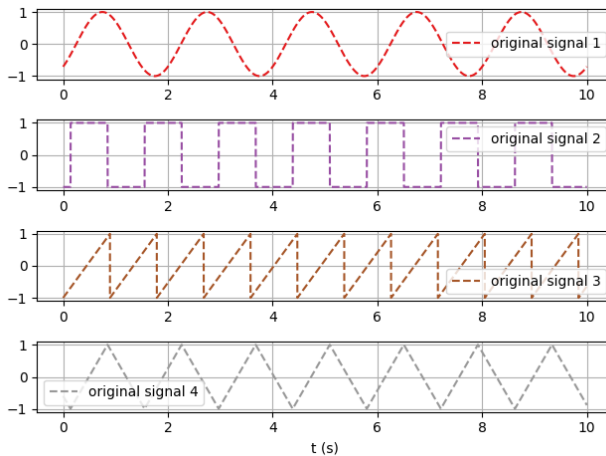
ICA in EEGLAB

FastICA in Python [1]

1. In this artificial example, 4 signals are generated: a sinusoidal signal, a square wave, a sawtooth wave, a triangular wave.
2. Note that the probability density function (pdf) of both sawtooth and triangular waves is uniform between the minimum and the maximum; the pdf of the square wave is discrete (two possible values: the minimum and the maximum), the pdf of the sinusoidal signal is $f(x) = \frac{1}{\pi\sqrt{x(1-x)}}$. Thus the pdfs are far from being Gaussian.
3. Each sample of the four signals is multiplied by a 4×4 square matrix (that is randomly generated), to generate four new mixed signals. Task of FastICA is to estimate the original signals from the new mixed signals.
4. In the Python code that you find in DropBox you have the generation of the original and mixed signals, with the measured/estimated pdfs.
5. You have the results obtained with FastICA, with plots.
6. Moreover, you have the results obtained with PCA, with plots.

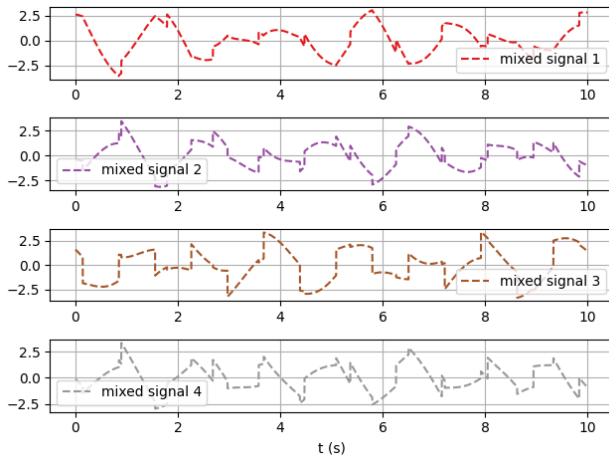
FastICA in Python [2]

Original signals



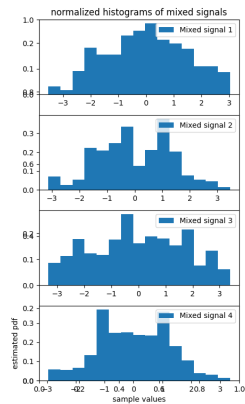
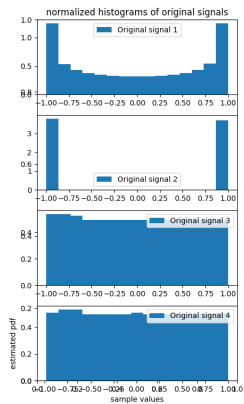
FastICA in Python [3]

Mixed signals



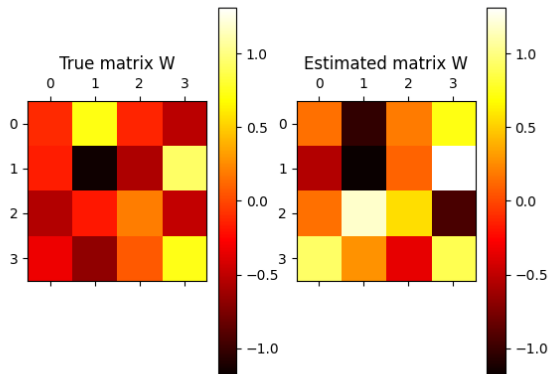
FastICA in Python [4]

Histograms



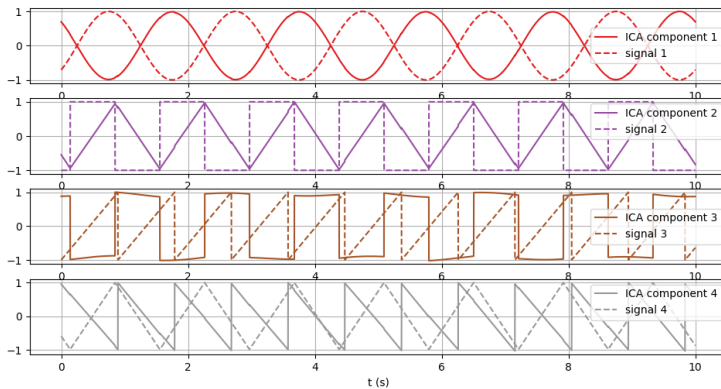
FastICA in Python [5]

FastICA estimation of matrix W



FastICA in Python [6]

Fast-ICA



FastICA in Python [7]

PCA

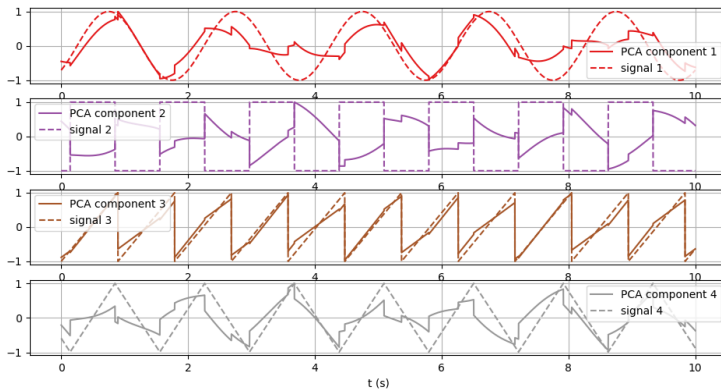


Table of Contents

Gaussianity tests

An artificial example for ICA

EEGLAB

ICA in EEGLAB

EEGLAB installation

EEGLAB is an add-on of MATLAB, therefore first you need to have MATLAB installed with the Signal Processing Toolbox installed.

You can (with some efferots) install EEGLAB from the home menu of MATLAB, by selecting Add-Ons, Get Add-Ons and searching for EEGLAB.

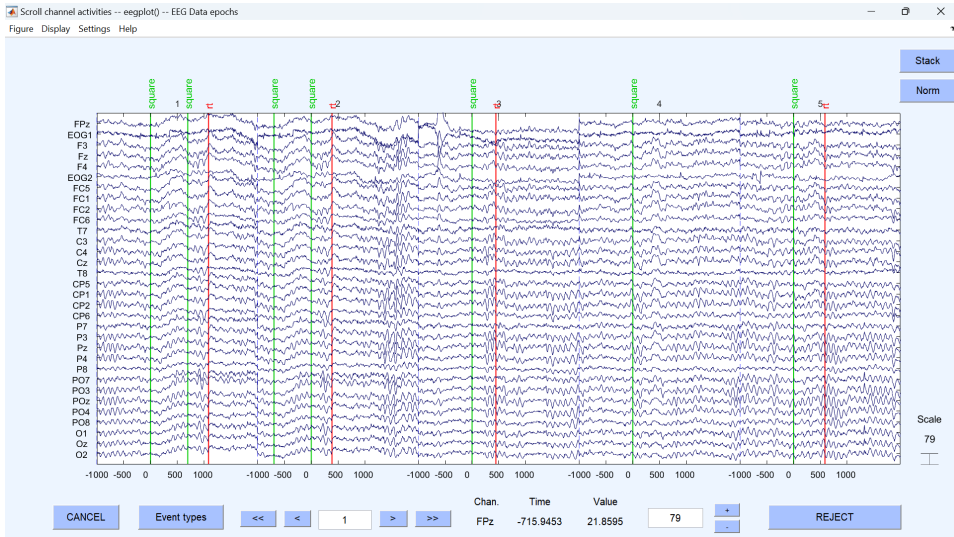
Otherwise you can download EEGLAB from the EEGLAB website

<https://sccn.ucsd.edu/eeglab/downloadtoolbox.php>, unzip the downloaded file in a folder, move with MATLAB in that folder (icon browse for folder in the home menu).

Starting EEGLAB [1]

1. From the MATLAB command window, simply write `eeglab` to start. A window will appear, with all the commands necessary to run EEGLAB.
2. The first step is to upload an EEG file. For this demo, we will use one of the available datasets: in the menu select File/Load existing dataset move to folder `sample_data` and then select `eeglab_data_epochs_ica.set`. The experiment was the following: the person was in front of a screen and had to click a button when he/she saw a square in one of 5 possible locations on the screen. In the plot of the next slide (Plot/Channel data (scroll)), you see the 32 EEG signals and green vertical lines corresponding to the times at which the square appeared and red vertical times when the button was pressed. Actually, what you see is the juxtaposition of 3 second clips: from one second before to two seconds after the appearance of the square on the screen. The number of experiments (epochs) is 80.

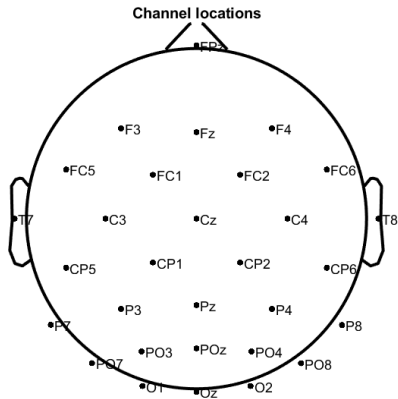
Starting EEGLAB [2]



Starting EEGLAB [3]

This dataset includes the locations of the electrodes, for other datasets locations should be provided separately. There are 32 electrodes, whose position you can see using Plot/Channel locations/By name

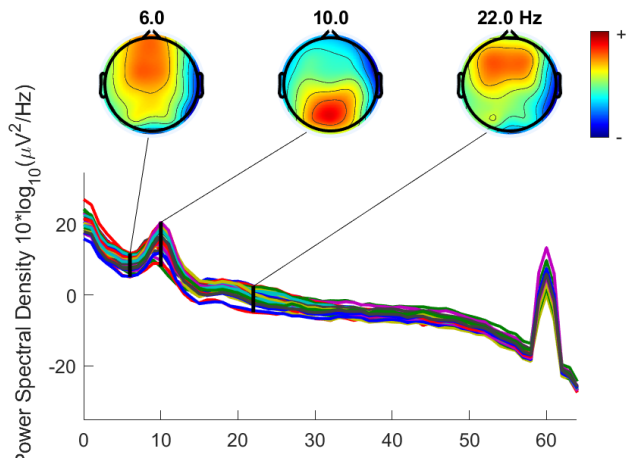
Starting EEGLAB [4]



Starting EEGLAB [5]

3. It is necessary to remove the linear trends and the interference from the mains, at 60 Hz. To see the presence of the interference, see the power spectra (i.e. how the signal power is distributed in frequency) using Plot/Channel Spectra and Maps and setting the frequency range to plot (5th parameter) as 0 64. In fact, the EEG samples were taken at sampling frequency 128 Hz and the maximum frequency that can be seen in the power spectrum is $128/2=64$ Hz. Note that each EEG signal in each “epoch” of 3 seconds corresponds to 384 samples. Power spectra are shown in dBs: $10 \log_{10}$ of the power spectrum. Students with no background on Fourier transforms are invited to listen to the videos in https://eeglab.org/tutorials/08_Plot_data/Time-Frequency_decomposition.html

Starting EEGLAB [6]



The meaning of the figure will be explained later on.

Starting EEGLAB [7]

To remove the linear trend, a high pass filter with corner frequency equal to 1 Hz can be used. To remove the mains interference you can use a notch filter (that only removes the 60 Hz component) or a low pass filter with bandwidth for example equal to 50 Hz. Note that neurologists identify (and are interested in):

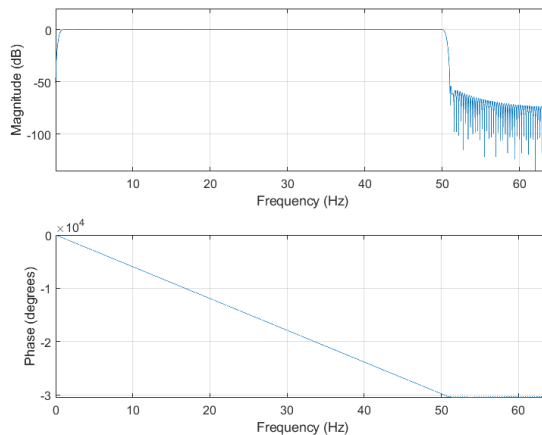
- ▶ δ waves (deep sleeping, not in this case) at 0.5-4 Hz
- ▶ θ waves (drowsiness) at 4-8 Hz
- ▶ α waves (relaxation; awake, but not processing information) at 8-13 Hz
- ▶ β waves (active thinking, problem solving) at 13-30 Hz
- ▶ γ waves (memory and learning, not in this case) at 30-60 Hz

Therefore, the resulting band pass filter, that allows to pass frequencies from 1 to 50 Hz, is correct.

To perform the filtering, use Tools/Filter the data/Basic FIR filter, and insert 1 and 50 as the first two parameters. Note for the students with a signal processing background: since reaction times are fundamental, it is necessary to use a filter that introduces a delay that is constant over the entire frequency range, and a linear phase FIR filter is therefore the correct choice. A new dataset is generated and a new name is asked for, for example EEG Data epochs Filtered

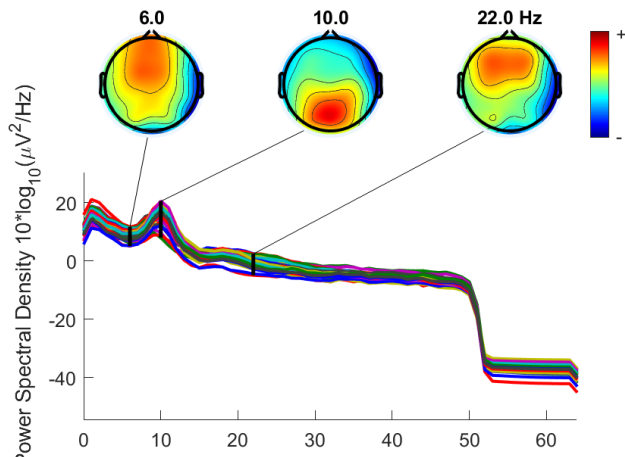
Starting EEGLAB [8]

Filter transfer function



Starting EEGLAB [9]

Power spectra of the signals after filtering.



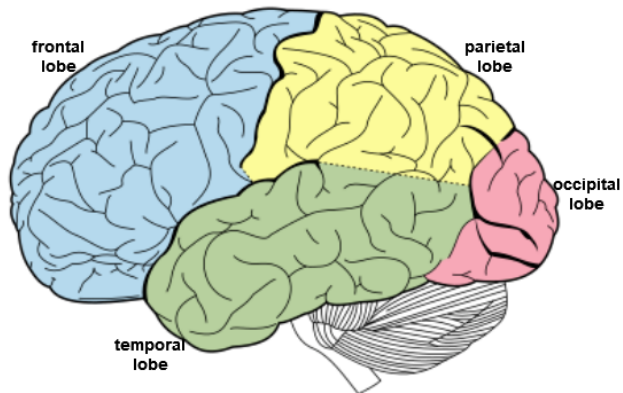
Starting EEGLAB [10]

Meaning of the the figure with power spectra:

- ▶ For each of the electrodes (at a known position on the skull) you measure the power spectrum.
- ▶ You plot the value of the power spectrum at selected frequencies (6, 10, 22 in the figure) in a 2-D image of the skull and using a color scale (you show, as color, the power spectrum value for each electrode, whose position is known), then you interpolate these values so that you color the entire 2D skull.
- ▶ θ waves (6 Hz) are more concentrated on the frontal lobe.
- ▶ α waves (10 Hz) are more concentrated on the parietal and occipital lobes.
- ▶ β waves (22 Hz) are more concentrated on the frontal lobe.

However, remember that the electrodes capture the sums of the signals generated inside the brain in different locations and therefore there is not such a strict correspondence between the locations shown in the power spectra plot and the true brain lobes involved in the task.

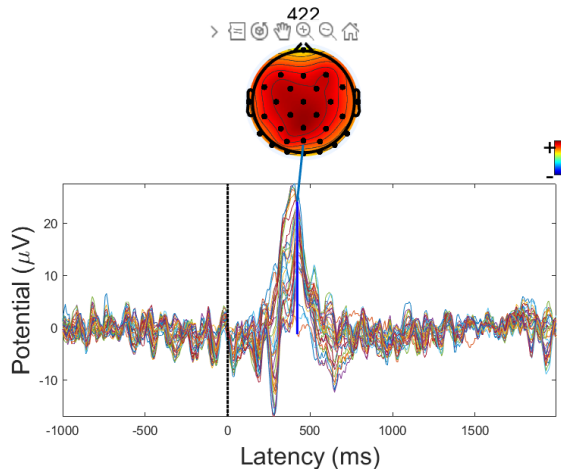
Starting EEGLAB [11]



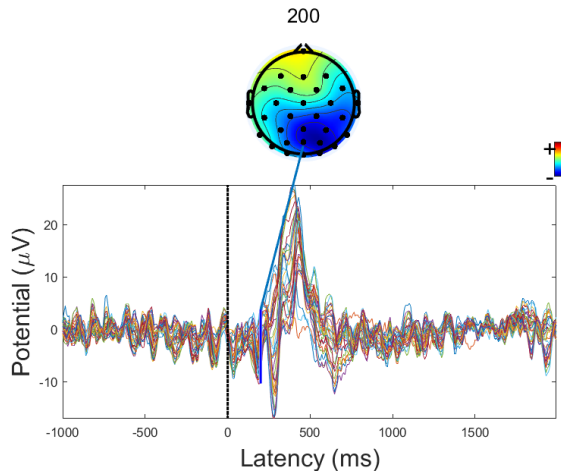
Other plots [1]

1. It is possible to view the ERPs (Event Related Potentials) that are obtained for each of the 32 electrodes by averaging the signals in each epoch (from -1 to 2 seconds around the appearance of the square on the screen): Plot/Channel ERPs/With scalp maps. In this case, the plot on the 2D scalp shows (in color scale) the amplitude of the averaged signal at a given latency (422 ms) from the appearance of the square on the screen. It is possible to change the latency by modifying the second input parameter from NaN to 200 (for example).

Other plots [2]



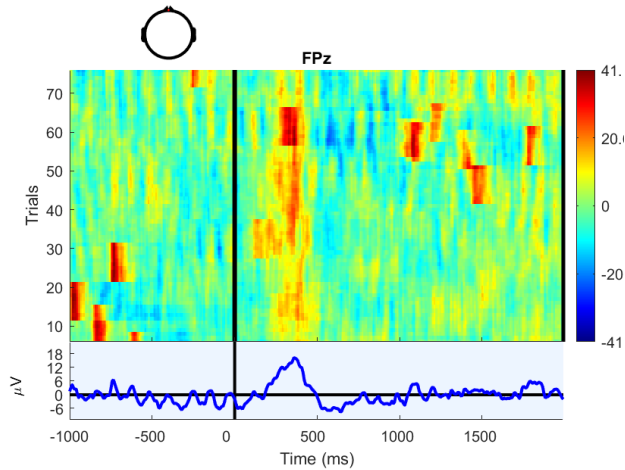
Other plots [3]



Other plots [4]

2. With Plot/Channel ERP Image, it is possible to see, for the selected electrode, the average ERP signal and, in color scale, each of the 80 signals in the experiment (out of which the average is evaluated). The electrode in the figure is number 1, middle front.

Other plots [5]



Other plots [6]

3. With Plot/Channel time-frequency the time-frequency plot is obtained for the selected sensor (again number 1). More details are given in the videos at https://eeglab.org/tutorials/08_Plot_data/Time-Frequency_decomposition.html, but, in a nutshell, the time-frequency plot is obtained by selecting a time window of samples (for example from sample 10 to sample 110), evaluate the Fast Fourier Transform (FFT) and plot the absolute values of the FFT as colors at the time corresponding to sample 10, then repeat the process on a subsequent time window (from sample 11 to sample 111), etc. In the next figure you see that at about 300 ms from the appearance of the square, signal for electrode FPz has mainly components at low frequencies (below 10 Hz, θ waves).

Other plots [7]

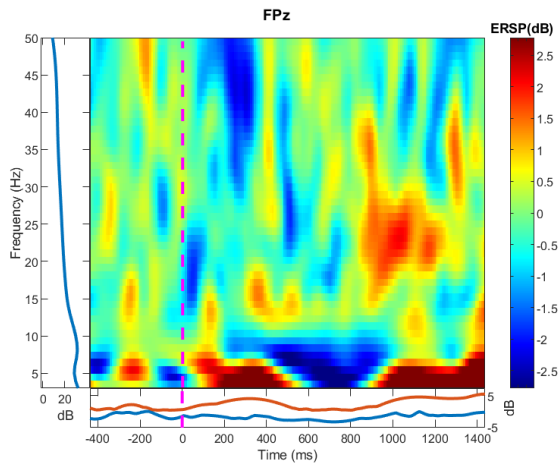


Table of Contents

Gaussianity tests

An artificial example for ICA

EEGLAB

ICA in EEGLAB

Independent Component Analysis [1]

1. Go to Tools/Decompose Data by ICA and leave the parameters unchanged. Independent Component Analysis starts (note that it is not FastICA, but Infomax ICA). All the 32 components (since there are 32 sensors) are generated in parallel using the gradient algorithm and maximizing the joint entropy (which corresponds to minimize the mutual information) after a nonlinear transformation (sigmoid) and quantization, more details in the videos:

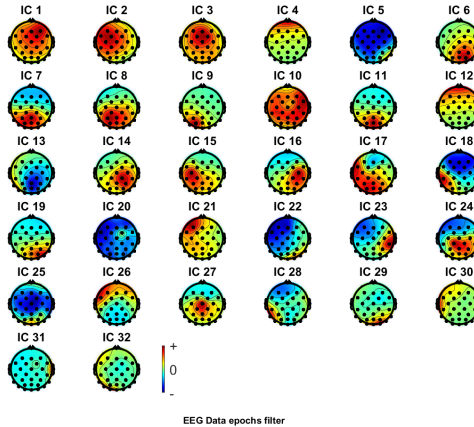
https://www.youtube.com/watch?v=kWAjhXr7pT4&list=PLXc9qfVbMMN2uDadxZ_OESHjzcRt1LNxc

Note that, while the algorithm runs, MATLAB shows in the command window the learning rate (which is decreased during the optimization process), the norm of the difference between the current matrix $\mathbf{W}$ and the previous one, and the angle between the current $\mathbf{W}$ and the previous one (basically, you evaluate the inner product, which can also be obtained as product of the norms times the cosine of the angle). If the angle is 180 degrees, it means that the algorithm is moving around the optimum solution, but with a too large learning coefficient and it is necessary to decrease it. In the implemented algorithm, if the angle is larger than 60 degrees, then the learning rate is multiplied by 0.98.

Independent Component Analysis [2]

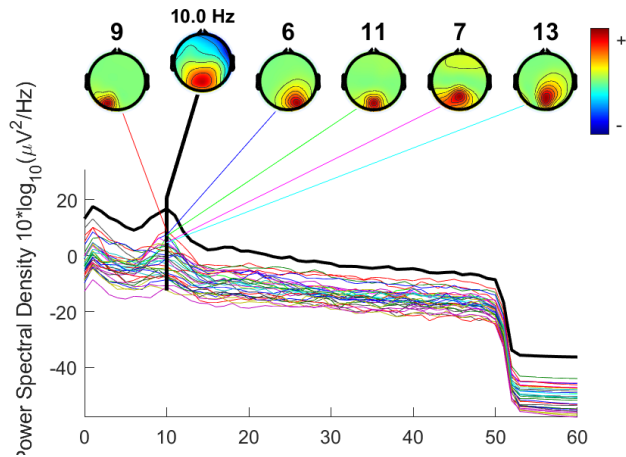
2. Once the components are found, it is possible to generate the plots.
 - ▶ Using Plot/Components maps/In 2D you can see the rows of matrix $\mathbf{W}$ that allow to generate each component from the original signals (remember that each original signal is associated with a location on the scalp).

Independent Component Analysis [3]



Independent Component Analysis [4]

- Using Plot/Components spectra and maps you can see how the overall 10 frequency power is divided into the various ICA components with associated map.



Independent Component Analysis [5]

- The most important point is that you can select components that are artifacts (like heart rate, eye blinking, etc). In the example, note component 4, which is almost always equal to zero, but shows a peak at time -750 ms in the middle of the plot (probably eye blinking). By removing artifacts, it is easier for the neurologist to interpret the EEG and diagnose illnesses.

